Agenda: Measures of Dispersion
— Variance

Percentile, IQR

Prob Distribution
Binomial Distribution

Variance

$$\frac{\sum (x - \bar{x})^2}{n}$$

$- , + \rightarrow 0$

large variance → more spread
small variance → less spread

Standard Deviation

$$\sigma = \sqrt{var}$$

data → cm → $\sqrt{cm^2}$

Percentile

A value below which a given percentage of observation in a dataset fall.

⟹ Relative position of a value in a dataset

50th Percentile — Median
25th Percentile — Q_1 (first Quartile)
75th Percentile — Q_3 (Third Quartile)

Quartiles

Min — Q_1 — Q_2 — Q_3 — Max

Q_1: 25th Q_2: Median 50th Q_3: 75th

Min - Q_1 — 25%

Q_1 - Q_2 — Median

Q_2 - Q_3 — 75%

Q_3 - Max

Interquartile Range

IQR

$Q_3 - Q_1$

Bottom & Top 25%

→ Outliers

10, 20, 30, 40, 50, 60, 70, 80, 90, 100

n = 10

Q_2 (Median) ⇒ $\frac{50+60}{2} = 55$

First part : Q_1 10, 20, 30, 40, 50 = 30

Second part : Q_3 ⇒ 60, 70, 80, 90, 100 = 80

$IQR = Q_3 - Q_1 = 80 - 30 = 50$

Outlier detection:

Lower Bound $Q_1 - 1.5 * IQR$
Upper Bound $Q_3 + 1.5 * IQR$

95th percentile → outperformed

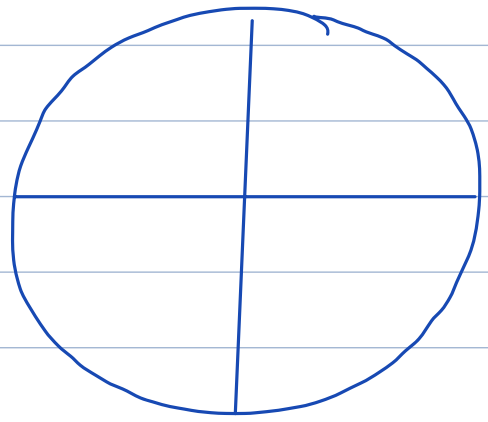
Salary distn.

50,000

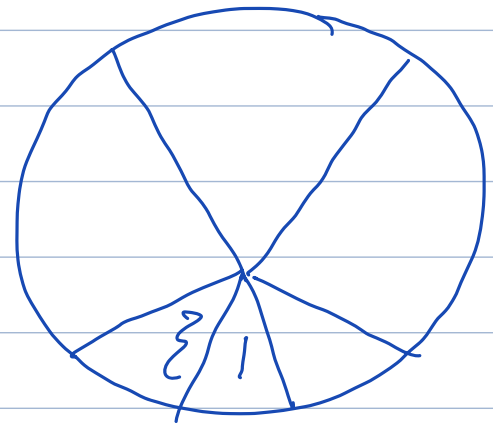
IQR ⇒ 40 – 60

or 20 – 1L

Probability Distribution



pizza
1/4



pizza

Which outcomes are more likely and which outcomes are less likely

P.D is a function that assigns a prob to each possible value of a random variable

$$P(X = x)$$

① $0 \leq P(X = x) \leq 1$

② $\sum P(X = x) = 1$

$P(\text{Head}) = 0.5$, $P(\text{Tail}) = 0.5$

$0.5 + 0.5 = 1$

Outcome $\longrightarrow$ Prob

Outcome	Prob
1	0.2
2	0.3
3	0.5
	1

Distribution

Discrete

Countable

- Dice
- Coin toss
- No. of student

$P(X=x)$

Continuous

Countable

- Ht
- Wt
- Time
- Marks

$f(x)$

Predictions $\longrightarrow$ Unequal distribution

Rain tomorrow = 30%
No rain = 70%

Predict $\Rightarrow$ No rain

$$P(X = a) \neq P(X = b)$$

Most likely outcome : $\operatorname{argmax}\left(P(X = x)\right)$

Binomial Distribution

If we repeat an experiment n no. of times, what is the prob. of getting exactly k successes?

$\Rightarrow$ n trials
2 possible outcomes
Prob stays the same

70% chance correctly
10 question

Expected value $= n * p$
$10 * 0.7$
$= 7$

$X : S \rightarrow R$

$X \sim Bin(n, p)$

n – trials
p – prob success
X – no. of successes in n trials

$$P(X=k) = C(n,k) \cdot p^k \cdot (1-p)^{n-k}$$

$$C(n,k) = \frac{n!}{k! * (n-k)!}$$

5 trials 3 heads

HHHTT

HTHTH

THHTH

$$\text{Mode} = \lfloor (n+1)\, p \rfloor$$

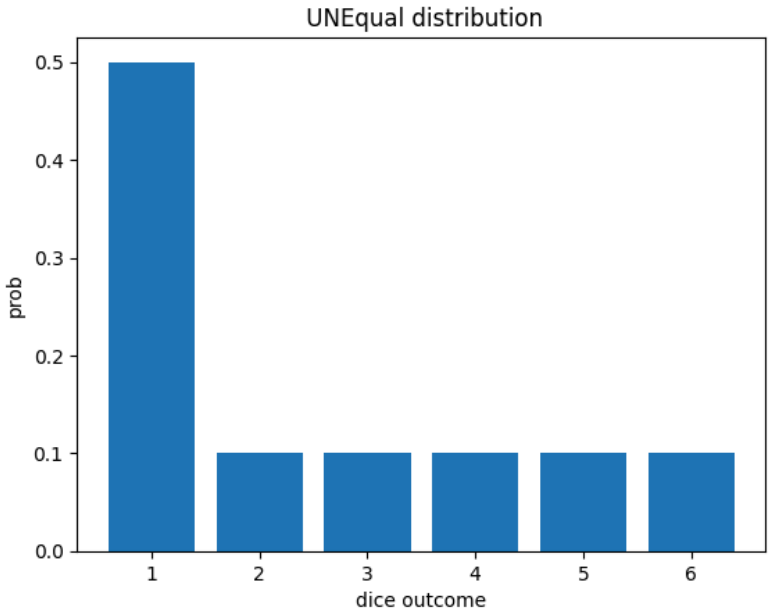
$$E(x) = n \cdot p$$

$$Var(X) = np(1-p)$$

$$\sigma = \sqrt{Var(X)}$$

```
# fair dice roll

# P(X=x)=1/6

import numpy as np
import matplotlib.pyplot as plt

outcomes=np.array([1,2,3,4,5,6])
fair_prob=np.array([1/6,1/6,1/6,1/6,1/6,1/6])

plt.figure()
plt.bar(outcomes, fair_prob)
plt.title("Fair Equal distribution")
plt.xlabel("dice outcome")
plt.ylabel("prob")
plt.show()
```

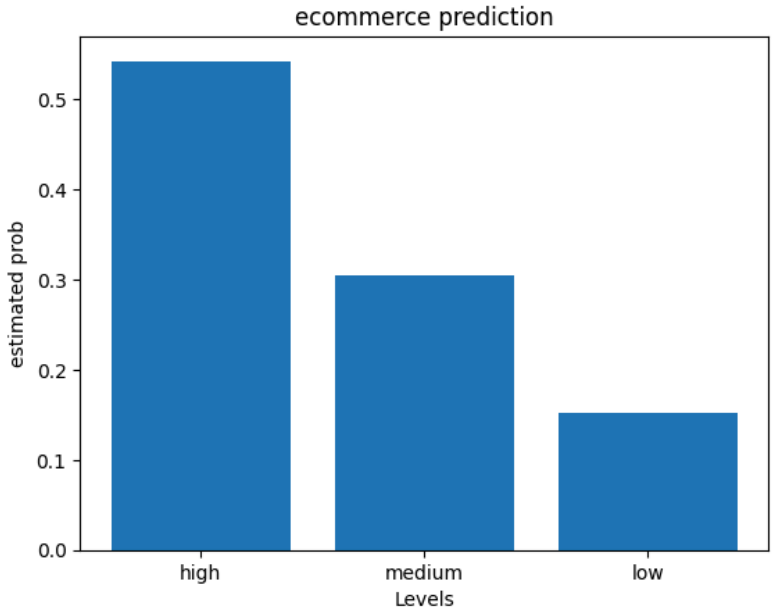
```
# unfair dice roll
# P(X=x)=not fixed

import numpy as np
import matplotlib.pyplot as plt

outcomes=np.array([1,2,3,4,5,6])
fair_prob=np.array([0.5,0.1,0.1,0.1,0.1,0.1])

plt.figure()
plt.bar(outcomes, fair_prob)
plt.title(" UNEqual distribution")
plt.xlabel("dice outcome")
plt.ylabel("prob")
plt.show()
```




```
# ecommerce
"""
Suppose an online store analyzed last 365 days data and found:
High Demand (Festivals, Big Sales) → around 55%

Medium Demand (Normal Weekdays) → around 30%

Low Demand (Off-season, Heavy Rain Days) → around 15%


Instead of directly plotting probabilities,
let’s simulate 10,000 future days based on historical trend.
"""
```

```
levels=["high","medium","low"]
prob=[0.55,0.30,0.15]

simulated_days=np.random.choice(levels, size=10000,p=prob)
print(simulated_days[:11])

counts=[] #[9,10,5]

for level in levels:
  comparison=(simulated_days==level) # 0 -> high -> comparison[0]=True
  count=np.sum(comparison) # 5
  counts.append(count) # counts[0]=5


est_prob=np.array(counts)/10000

plt.figure()
plt.bar(levels,est_prob)
plt.title("ecommerce prediction")
plt.xlabel("Levels")
plt.ylabel("estimated prob")
plt.show()

most_likely=levels[np.argmax(est_prob)]
print(most_likely)

print(counts)
```

```
['high' 'high' 'low' 'high' 'medium' 'low' 'medium' 'medium' 'low' 'high'
 'high']
```



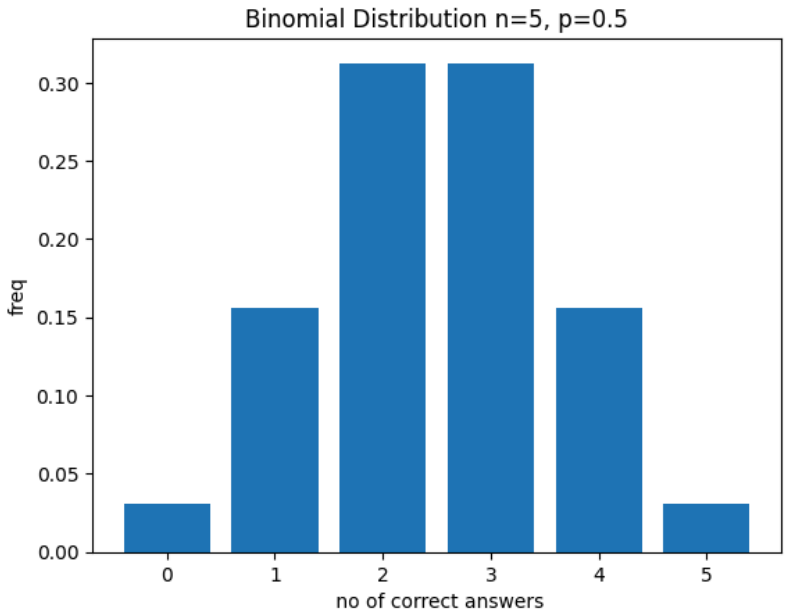

```
high
[np.int64(5426), np.int64(3054), np.int64(1520)]
```

```
"""
A student attempts a 5-question MCQ quiz.
Each question:
Has only one correct answer

The student guesses randomly

Probability of getting a question correct = 0.5


Let
X=Number of correct answers
What is the probability of getting 0,1,2,3,4,5 correct answers?

Verify that total probability = 1.

Plot the probability distribution.
"""
n=10
p=0.5
```

```
from math import comb

n=5
p=0.5

k_values=np.arange(0,n+1) #[0,1,2,3,4,5]
probability=[]

# apply binomial formula
# P(X=k) = nCk * p^k * (1-p)^n-k

for k in k_values:
  combination=comb(n,k) # nCk
  prob= combination * (p**k) * ((1-p)**(n-k))
  probability.append(prob)

print("probabilioty of each value of X")

for i in range(len(k_values)):
  print("P(X=",k_values[i],")=",probability[i])

print(sum(probability))

plt.figure()
plt.bar(k_values,probability)
plt.title("Binomial Distribution n=5, p=0.5")
plt.xlabel(" no of correct answers")
```

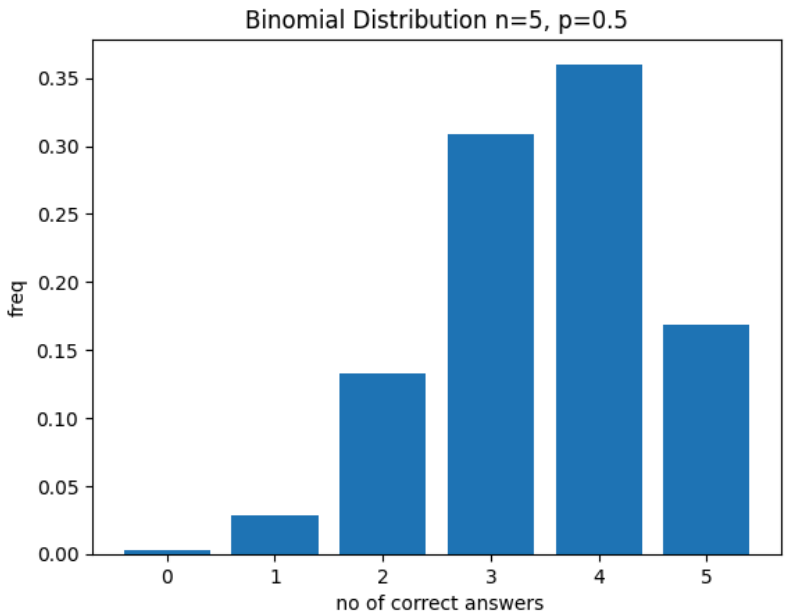
```
plt.ylabel("freq")
plt.show()
```

```
probabilioty of each value of X
P(X= 0 )= 0.03125
P(X= 1 )= 0.15625
P(X= 2 )= 0.3125
P(X= 3 )= 0.3125
P(X= 4 )= 0.15625
P(X= 5 )= 0.03125
1.0
```



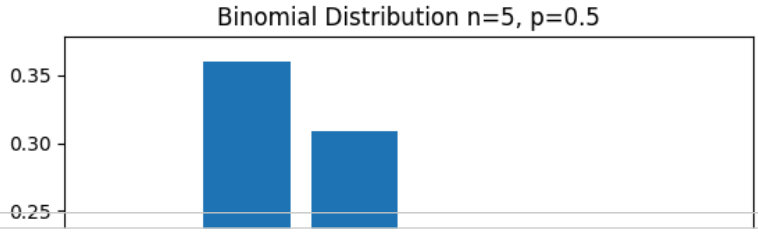

```
from math import comb

n=5
p=0.7

k_values=np.arange(0,n+1) #[0,1,2,3,4,5]
probability=[]

# apply binomial formula
# P(X=k) = nCk * p^k * (1-p)^n-k

for k in k_values:
  combination=comb(n,k) # nCk
  prob= combination * (p**k) * ((1-p)**(n-k))
  probability.append(prob)

print("probabilioty of each value of X")

for i in range(len(k_values)):
  print("P(X=",k_values[i],")=",probability[i])

print(sum(probability))

plt.figure()
plt.bar(k_values,probability)
plt.title("Binomial Distribution n=5, p=0.5")
plt.xlabel(" no of correct answers")
plt.ylabel("freq")
plt.show()
```

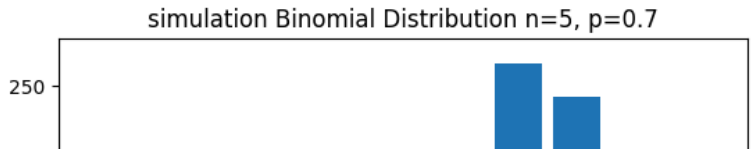
```
probabilioty of each value of X
P(X= 0 )= 0.0024300000000000016
P(X= 1 )= 0.028350000000000018
P(X= 2 )= 0.13230000000000006
P(X= 3 )= 0.30870000000000003
P(X= 4 )= 0.36014999999999997
P(X= 5 )= 0.16806999999999994
1.0
```




```
from math import comb

n=5
p=0.3

k_values=np.arange(0,n+1) #[0,1,2,3,4,5]
probability=[]

# apply binomial formula
# P(X=k) = nCk * p^k * (1-p)^n-k

for k in k_values:
  combination=comb(n,k) # nCk
  prob= combination * (p**k) * ((1-p)**(n-k))
  probability.append(prob)

print("probabilioty of each value of X")

for i in range(len(k_values)):
  print("P(X=",k_values[i],")=",probability[i])

print(sum(probability))

plt.figure()
plt.bar(k_values,probability)
plt.title("Binomial Distribution n=5, p=0.5")
plt.xlabel(" no of correct answers")
plt.ylabel("freq")
plt.show()
```

```
probabilioty of each value of X
P(X= 0 )= 0.16806999999999994
P(X= 1 )= 0.3601499999999999
P(X= 2 )= 0.3086999999999999
P(X= 3 )= 0.13229999999999997
P(X= 4 )= 0.028349999999999993
P(X= 5 )= 0.0024299999999999994
0.9999999999999999
```




```
import random
n=10
p=0.7

results=[]

for _ in range(1000):
  success=0
  for _ in range(n):
    if random.random()<p:
      success+=1
  results.append(success)

print(results[:10])

#[0,1,0,0,0,0,0,0,2,1,1,3]
# count how many students got 0,1,2
counts=[results.count(k) for k in range(n+1)]
# [7,3,1,1]
print(counts[:10])


plt.figure()
plt.bar(range(n+1), counts)
plt.title("simulation Binomial Distribution n=5, p=0.7")
plt.xlabel(" no of correct answers")
plt.ylabel("freq")
plt.show()
```

```
[7, 6, 5, 8, 9, 8, 9, 7, 7, 6]
[0, 0, 2, 7, 33, 110, 202, 262, 244, 117]
```